

Denormalization

If our service's data can fit into a single host, a common approach is to use an SQL database with a **normalized schema**. However, to improve read performance, many systems use **denormalization**, which means storing duplicate or pre-joined data to reduce expensive JOIN queries.

1. What is Denormalization?

Denormalization is the process of intentionally adding redundant data into database tables so queries become faster and simpler.

Instead of:

None

Orders table → customer_id

Customers table → customer_name

Use:

None

Orders table → customer_id, customer_name

Now queries do not need JOIN.

2. Why It Matters

JOIN operations become slower when:

None

Large tables

High traffic systems

Multiple joins

Distributed/sharded databases

Frequent read requests

Denormalization improves read speed.

3. Benefits of Normalization

A normalized schema provides:

None

No duplicate data

Better consistency

Smaller storage size

Faster inserts/updates

Fewer indexes

Cleaner schema design

Example:

None

Customer name stored once

4. Problems with Normalization

Normalized schemas often require JOINS:

None

Orders JOIN Customers

Orders JOIN Products

Orders JOIN Payments

This causes:

None

Slow reads

Complex SQL queries

Higher CPU usage

More disk I/O

Harder analytics queries

5. Why Denormalization is Used

To trade storage for speed.

Instead of expensive JOIN:

None

```
SELECT * FROM Orders JOIN Customers
```

Use:

None

```
SELECT * FROM Orders
```

6. Example

Normalized Schema

None

Customers:

1, Samarth

Orders:

101, customer_id=1

Need JOIN to get customer name.

Denormalized Schema

None

Orders:

101, customer_id=1, customer_name=Samarth

Single table query only.

7. Where Denormalization is Common

Use in:

- E-commerce product pages
- Dashboards
- Reporting systems
- Analytics tables
- Social feeds
- Search indexes
- Read-heavy APIs

8. HLD Example

None

Users Table

Orders Table

Products Table

For order history API:

Instead of 3 JOINS, create:

None

OrderSummary Table

order_id

user_name

product_name

price

status

Fast reads.

9. Read vs Write Tradeoff

Operation	Normalized	Denormalized
Read Speed	Slower	Faster
Write Speed	Faster	Slower
Storage	Smaller	Larger
Consistency	Easier	Harder

10. Write Complexity in Denormalization

If customer changes name:

None

Samarth → Samarth P

Need update in:

None

Customers table

Orders table

Invoices table

Reports table

This increases maintenance cost.

11. Storage Tradeoff

Duplicate data increases storage:

None

customer_name repeated in millions of orders

Acceptable when read speed is priority.

12. Sharded Systems

JOINS across shards are expensive.

Denormalization helps because:

None

Each shard contains complete row data

Useful in distributed systems.

13. Common Patterns

A. Precomputed Summary Table

None

DailySalesSummary

B. Materialized View

Pre-joined stored result.

C. Embedded JSON

Store child records inside parent row.

14. Example Query Benefit

Without denormalization:

None

```
JOIN Orders + Users + Products + Payments
```

With denormalization:

None

```
SELECT * FROM order_summary
```

Much faster.

15. Risks

None

Data inconsistency

More update logic

Extra storage

Harder schema changes

Sync delays

16. Best HLD Practice

Use:

None

Normalized OLTP database

+

Denormalized read models

+

Cache layer

Example:

None

MySQL + Redis + Summary Tables

17. Interview Answer

If reads are slow due to JOINS:

None

Use denormalized tables

Precompute common joins

Use async pipelines to sync data

Cache frequent queries

Strong answer.

18. When to Avoid

Avoid if system is:

None

Write-heavy

Strict consistency required

Frequent updates

Small dataset

19. Advantages

None

Fast reads

Simple queries

Better scalability

Good for dashboards

Better user response time

20. Disadvantages

None

Duplicate data

Higher storage

Complex writes

Possible stale data

Harder maintenance

21. One-Line Summary

Denormalization intentionally duplicates data to reduce JOIN operations and improve read performance, trading storage efficiency and write simplicity for speed and scalability.